Guía Práctica para Estudiantes: Base de Datos Relacional con MySQL (E-Commerce)

1. Presentación y Objetivo del Proyecto

¡Bienvenido/a a tu guía práctica de base de datos! En este laboratorio práctico aprenderás a diseñar, construir y consultar una base de datos relacional completa para una empresa de comercio electrónico llamada **"TechStore S.A."**.

A lo largo de este proyecto aprenderás a:

- Definir estructuras de tablas relacionales mediante comandos **DDL** (`CREATE`, `DROP`, `ALTER`).
- Aplicar restricciones de integridad referencial y de dominio (**`PRIMARY KEY`**, **`FOREIGN KEY`**, **`UNIQUE`**, **`NOT NULL`**, **`CHECK`**, **`DEFAULT`**, **`AUTO_INCREMENT`**).
- Definir reglas de borrado referencial (**`ON DELETE CASCADE`** y **`ON DELETE SET NULL`**).
- Manipular información almacenada mediante comandos **DML** (`INSERT`, `UPDATE`, `DELETE`).
- Consultar y filtrar información mediante comandos **DQL** (`SELECT`, `WHERE`, `ORDER BY`, `LIKE`).
- Realizar combinaciones de múltiples tablas mediante consultas con **`INNER JOIN`**, **`LEFT JOIN`** y **`RIGHT JOIN`**.

2. Caso de Uso: "TechStore S.A."

Contexto

La empresa **"TechStore S.A."** necesita migrar su gestión de ventas a una base de datos relacional en MySQL para solucionar problemas de inconsistencias de inventario, pérdida de datos de clientes y la falta de reportes unificados de ventas.

Requerimientos de la Base de Datos

1. **Clientes:** Guardar el nombre, apellido, correo electrónico (único e indispensable) y teléfono de cada comprador.
2. **Categorías y Productos:** Organizar los productos por categorías. Un producto pertenece a una categoría opcional (si la categoría se elimina, el producto debe conservar su registro pero con categoría nula). Los precios deben ser estrictamente mayores o iguales a cero y el stock predeterminado debe ser cero.
3. **Pedidos:** Registrar los pedidos realizados por los clientes. Si un cliente es eliminado del sistema, por normativas de privacidad todos sus pedidos asociados deben eliminarse automáticamente en cascada.
4. **Detalle de Compra (Relación N:M):** Permitir que un pedido contenga múltiples productos y que un producto esté en múltiples pedidos, guardando la cantidad comprada (debe ser mayor a cero) y el precio unitario al que se vendió en ese instante.
5. **Consultas e Informes:** Consultar productos sin categoría, categorías sin productos, clientes sin compras e historial completo de cada pedido.

3. Guía de Conceptos Teóricos Clave

A. Sublenguajes de SQL

- **DDL (Data Definition Language):** Comandos para crear o modificar estructuras de objetos (`CREATE`, `ALTER`, `DROP`, `TRUNCATE`).
- **DML (Data Manipulation Language):** Comandos para insertar, modificar o eliminar registros (`INSERT`, `UPDATE`, `DELETE`).
- **DQL (Data Query Language):** Comandos para consultar datos (`SELECT`).

B. Restricciones e Integridad Referencial

- **`PRIMARY KEY`:** Identificador único e irrepetible para cada registro.
- **`FOREIGN KEY`:** Campo que establece un enlace de integridad con la clave primaria de otra tabla.
- **`ON DELETE CASCADE`:** Al borrar el registro de la tabla padre, borra automáticamente los registros relacionados en la tabla hija.
- **`ON DELETE SET NULL`:** Al borrar el registro de la tabla padre, asigna el valor `NULL` a la clave foránea en la tabla hija.

- **`CHECK`:** Condición lógica de validación obligatoria antes de insertar o modificar un dato.
- **`UNIQUE`:** Prohíbe valores duplicados en una columna.

C. Tipos de Combinaciones (`JOINS`)

- **`INNER JOIN`:** Retorna únicamente los registros que coinciden en ambas tablas.
- **`LEFT JOIN`:** Retorna todos los registros de la tabla izquierda y las coincidencias de la derecha (completa con `NULL` si no hay relación).
- **`RIGHT JOIN`:** Retorna todos los registros de la tabla derecha y las coincidencias de la izquierda (completa con `NULL` si no hay relación).

4. Guion del Ejercicio Práctico Paso a Paso

Abre tu cliente MySQL (MySQL Workbench, phpMyAdmin o terminal) y ejecuta las sentencias script por script:

PASO 1: DDL — Creación de la Base de Datos y Tablas

```
-- 1. Crear y seleccionar la base de datos

DROP DATABASE IF EXISTS tienda_online;

CREATE DATABASE tienda_online;

USE tienda_online;

-- 2. Tabla Clientes (Demuestra UNIQUE, NOT NULL, PRIMARY KEY y AUTO_INCREMENT)

CREATE TABLE clientes (

    cliente_id INT AUTO_INCREMENT,

    nombre VARCHAR(50) NOT NULL,

    apellido VARCHAR(50) NOT NULL,

    email VARCHAR(100) NOT NULL UNIQUE,
```

```
    telefono VARCHAR(20),
    PRIMARY KEY (cliente_id)
);

-- 3. Tabla Categorías
CREATE TABLE categorias (
    categoria_id INT AUTO_INCREMENT,
    nombre_categoria VARCHAR(50) NOT NULL,
    PRIMARY KEY (categoria_id)
);

-- 4. Tabla Productos (Demuestra CHECK, DEFAULT y FOREIGN KEY con ON DELETE SET NULL)
CREATE TABLE productos (
    producto_id INT AUTO_INCREMENT,
    nombre_producto VARCHAR(100) NOT NULL,
    precio DECIMAL(10, 2) NOT NULL CHECK (precio >= 0),
    stock INT NOT NULL DEFAULT 0,
    categoria_id INT,
    PRIMARY KEY (producto_id),
    CONSTRAINT fk_productos_categorias
        FOREIGN KEY (categoria_id) REFERENCES categorias(categoria_id)
        ON DELETE SET NULL
```

```
);

-- 5. Tabla Pedidos (Demuestra FOREIGN KEY con ON DELETE CASCADE)
CREATE TABLE pedidos (
    pedido_id INT AUTO_INCREMENT,
    fecha_pedido DATETIME DEFAULT CURRENT_TIMESTAMP,
    estado VARCHAR(20) DEFAULT 'Pendiente',
    cliente_id INT,
    PRIMARY KEY (pedido_id),
    CONSTRAINT fk_pedidos_clientes
        FOREIGN KEY (cliente_id) REFERENCES clientes(cliente_id)
        ON DELETE CASCADE
);

-- 6. Tabla Detalle_Pedidos (Clave Primaria Compuesta para relación N:M)
CREATE TABLE detalle_pedidos (
    pedido_id INT,
    producto_id INT,
    cantidad INT NOT NULL CHECK (cantidad > 0),
    precio_unitario DECIMAL(10, 2) NOT NULL,
    PRIMARY KEY (pedido_id, producto_id),
    CONSTRAINT fk_detalle_pedidos
```

```
    FOREIGN KEY (pedido_id) REFERENCES pedidos(pedido_id),
  CONSTRAINT fk_detalle_productos
    FOREIGN KEY (producto_id) REFERENCES productos(producto_id)
);
```

PASO 2: DML — Inserción, Actualización y Eliminación

```
-- 1. INSERT: Categorías
INSERT INTO categorias (nombre_categoria) VALUES
('Electrónica'),
('Ropa'),
('Hogar'),
('Jardín'); -- Categoría sin productos para probar RIGHT JOIN

-- 2. INSERT: Productos
INSERT INTO productos (nombre_producto, precio, stock, categoria_id) VALUES
('Laptop Gamer', 1200.00, 10, 1),
('Smartphone', 800.00, 15, 1),
('Camiseta Algodón', 25.00, 50, 2),
('Lámpara de Mesa', 45.00, 20, 3),
('Mouse Inalámbrico', 30.00, 5, NULL); -- Producto sin categoría para probar LEFT JOIN

-- 3. INSERT: Clientes
INSERT INTO clientes (nombre, apellido, email, telefono) VALUES
```

('Carlos', 'Mendoza', 'carlos@email.com', '555-0101'),

('Ana', 'García', 'ana@email.com', '555-0102'),

('Luis', 'Torres', 'luis@email.com', '555-0103'),

('María', 'Reyes', 'maria@email.com', '555-0104'); -- Cliente sin pedidos para probar LEFT JOIN

-- 4. INSERT: Pedidos y Detalles de Compra

INSERT INTO pedidos (cliente_id, estado) VALUES

(1, 'Completado'),

(1, 'En Proceso'),

(2, 'Pendiente');

INSERT INTO detalle_pedidos (pedido_id, producto_id, cantidad, precio_unitario) VALUES

(1, 1, 1, 1200.00),

(1, 3, 2, 25.00),

(2, 2, 1, 800.00),

(3, 4, 1, 45.00);

-- 5. UPDATE: Modificar datos de productos

UPDATE productos

SET stock = stock + 10, precio = 28.00

WHERE producto_id = 3;

-- 6. DELETE: Probar la eliminación en cascada (ON DELETE CASCADE)

```
DELETE FROM clientes WHERE cliente_id = 3;
```

PASO 3: DQL — Consultas y Filtros Básicos

```
-- Consultar todos los registros de clientes

SELECT * FROM clientes;

-- Filtrar productos con precio mayor a $50 ordenados por precio descendente

SELECT nombre_producto, precio, stock

FROM productos

WHERE precio > 50.00

ORDER BY precio DESC;

-- Filtrar clientes por coincidencia de texto mediante LIKE

SELECT * FROM clientes WHERE email LIKE '%@email.com';
```

PASO 4: DQL Avanzado — Consultas con JOINS

A. INNER JOIN (Coincidencias de ambas tablas)

```
SELECT

    p.nombre_producto,

    p.precio,

    c.nombre_categoria

FROM productos p

INNER JOIN categorias c ON p.categoria_id = c.categoria_id;
```

B. LEFT JOIN (Todos los registros de la izquierda + coincidencias de la derecha)

```
-- Muestra todos los productos, incluyendo el Mouse que no tiene categoría

SELECT
```

```
    p.nombre_producto,
    p.precio,
    IFNULL(c.nombre_categoria, 'Sin Categoría') AS categoria
FROM productos p
LEFT JOIN categorias c ON p.categoria_id = c.categoria_id;
-- Muestra los clientes que NO han hecho ningún pedido
SELECT c.nombre, c.apellido
FROM clientes c
LEFT JOIN pedidos p ON c.cliente_id = p.cliente_id
WHERE p.pedido_id IS NULL;
```

C. RIGHT JOIN (Todos los registros de la derecha + coincidencias de la izquierda)

```
-- Muestra todas las categorías, incluyendo 'Jardín' que no tiene productos
SELECT
    c.nombre_categoria,
    p.nombre_producto
FROM productos p
RIGHT JOIN categorias c ON p.categoria_id = c.categoria_id;
```

D. Consulta Integradora Completa (Múltiples JOINs)

```
SELECT
    p.pedido_id,
    CONCAT(c.nombre, ' ', c.apellido) AS cliente,
```

```
    pr.nombre_producto,
    dp.cantidad,
    dp.precio_unitario,
    (dp.cantidad * dp.precio_unitario) AS subtotal,
    p.estado
FROM pedidos p
INNER JOIN clientes c ON p.cliente_id = c.cliente_id
INNER JOIN detalle_pedidos dp ON p.pedido_id = dp.pedido_id
INNER JOIN productos pr ON dp.producto_id = pr.producto_id;
```

5. Cuestionario de Autoevaluación

Responde a las siguientes preguntas para verificar tu aprendizaje:

1. **¿Qué diferencia existe entre `DELETE FROM tabla` y `TRUNCATE TABLE tabla`?**
    - *Respuesta:* `DELETE` es una sentencia DML que elimina filas una a una según una condición (`WHERE`), conservando los contadores de `AUTO_INCREMENT`. `TRUNCATE` es una sentencia DML/DDL que borra instantáneamente todos los registros y reinicia el contador de `AUTO_INCREMENT` a 1.
2. **¿Por qué se requiere una clave primaria compuesta en la tabla `detalle_pedidos`?**
    - *Respuesta:* La combinación de `(pedido_id, producto_id)` asegura que no se pueda agregar el mismo producto más de una vez en un mismo pedido y permite representar la relación muchos a muchos (N:M).
3. **¿Qué ocurre con los registros de la tabla `pedidos` cuando se elimina un registro en la tabla `clientes`?**

- *Respuesta:* Debido a la regla `ON DELETE CASCADE` en la clave foránea `fk_pedidos_clientes`, todos los pedidos realizados por ese cliente se eliminan automáticamente de la base de datos.